

The Data-Centric Architecture

Semantic Arts

Data-Centric Architecture

1

Data-Centricity

Data-Centric Architecture

2

2

What is Data-Centricity?

- In the data-centric world, there is a single, persistent, evolving, shared, open data model
 - Not necessarily a single database
 - But a single shared schema
- The applications are transient, all built around the shared schema for their inputs, parameters, and outputs

Data-Centric Architecture

3

3

Data-Driven vice Data-Centric

- Data-Driven organizations...
 - ...use data as the basis for decisions
 - ...tend to use data scientists and Big Data-style analytics
- But...
 - ...there is nothing inherent in the Data-Driven approach that leads to Data-Centric style consolidation or rationalization
 - ...the analytical results are not necessarily retained and fed back into the data store

Data-Centric Architecture

4

4

Data-Centric App vice Data-Centric Architecture

- An application can be data-centric if...
 - It stores its parameters in a single, shared datastore,
 - It pulls its inputs from that same shared datastore, and
 - It stores its outputs in that same shared datastore
- But a data-centric *application* does not require or imply the existence of a data-centric *architecture*

- A Data-Centric Architecture describes a whole environment, an ecosystem

Data-Centric Architecture

5

5

A Data-Centric Architecture

(Dare we say The DCA)

Data-Centric Architecture

6

6



Data-Centric Architecture Characteristics

- Recall: the data-centric environment is built around *a single, persistent, evolving, shared, open data model*
- To be used in a data-centric fashion, that model should have these characteristics:
 - Open Standards
 - Self-Defining Data
 - Flexibility
 - Retained Algorithmic Results
 - Data Integrity
 - Simplicity
 - High Fidelity
 - Security
 - Provenance
 - Extensibility
 - Federability

Data-Centric Architecture

7

7



DCA & Open Standards

- All data and metadata must be captured in open standard formats
 - You should be able to access and interpret the data directly
- This is vital to avoiding any proprietary vendor lock-in

Data-Centric Architecture

8

8

DCA & Self-Definition

- You should not rely on application code or a specification somewhere outside the database to interpret what any item of data means
- All meaning (and anything needed to interpret the data) needs to be in the same database*
 - And cross-referenced with the data or metadata to which it refers

**Technically, it can be in multiple databases so long as they are federated, meaning a single query can access any and all of them*

But it is quicker to say "the same"

DCA & Flexibility

- Your data environment will change
- The data-centric structures need to be flexible
- They should accommodate new data without invalidating existing data
 - And without taking weeks and months to do so

DCA & Algorithmic Results Data

- Algorithmic results should be persisted in the database, in a manner independent of the algorithm
- Example: If a reorder routine runs and suggests orders, the orders should be persisted in the database
 - Any procurement functionality could pick them up and run with them without knowing the details of how the reorder routine worked

Data-Centric Architecture

11

11

DCA & Data Integrity

- The data architecture should define and manage its own data integrity
- Data integrity rules should be expressed in a syntax that a user could understand
 - Although they shouldn't need to
- The data integrity rules should be queryable / discoverable
- The system should manage the data integrity whether users understand it or not

Data-Centric Architecture

12

12

DCA & Simplicity

- The data model should be simple enough that a motivated user could understand the whole of it
- The model should be understandable enough such that a user can predict which parts of the data they need to access in order to accomplish what they want

Data-Centric Architecture

13

13

DCA & Fidelity

- The data model must make all the distinctions the business needs,...
- ...at the level of detail that the business needs them...
- ...and correctly reflect the multiple contexts in which the data are used
- The model cannot be simplified to the point where any important business process cannot be satisfied

Data-Centric Architecture

14

14

DCA & Security

- The data architecture must have a robust, fine-grained approach to
 - authentication,
 - authorization, and
 - user privileges
- But it cannot rely on application-supplied logic to accomplish this
- The parametric data for authentication, etc., must be stored and queried from the datastore

Data-Centric Architecture

15

15

DCA & Provenance

- The architecture must support the means for recording and querying metadata about particular datasets
 - Where did it come from?
 - How old is it?
 - Who owns it?
- This enables users to include or exclude those datasets from their processing, based on their level of trust in the data

Data-Centric Architecture

16

16

DCA & Extensibility

- The model must allow organizations to incorporate it into their own specialized models (while remaining unchanged, itself)
- Example: A model might define the *Inventory* domain, generically
 - One extension defines the *Hand Tools Inventory*
 - Another defines the *Fresh Produce Inventory*
 - Each includes the additional attributes necessary for its domain

Data-Centric Architecture

17

17

DCA & Federability

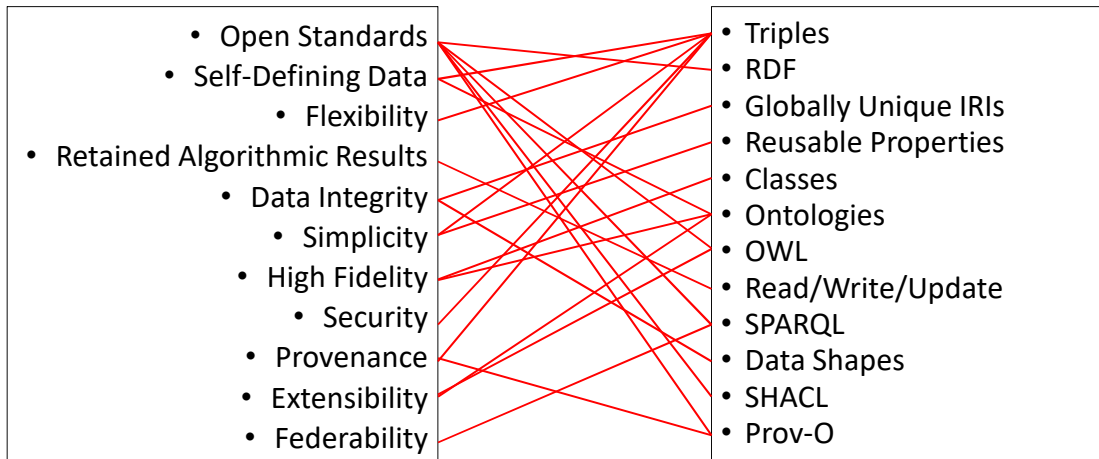
- The architecture must support federating queries over multiple repositories, if needed
- The architecture must allow each repository to have different levels of security and/or integrity management

Data-Centric Architecture

18

18

DCA Characteristics and Semantic Tech



Data-Centric Architecture

19

19

Non-Semantic Approaches to Data-Centricity

- Must you use semantic technology to implement a Data-Centric approach?
 - No
 - We know of companies that have Data-Centric implementations using traditional relational databases
- But...
 - One of those companies is realizing they are approaching the limits of the flexibility that relational DBs give them
 - They are exploring a transition to semantic tech
 - The reusability of IRI-based semantic entities makes for smaller, less-complex data-centric environments

Data-Centric Architecture

20

20

Data-Centricity, redux

- In the data-centric world, there is a single, persistent, evolving shared open data model
 - Not necessarily a single database
 - But a single shared schema
- The applications are transient, all built around the shared schema for their inputs, parameters, and outputs
- A Data-Centric Architecture describes a whole environment, an ecosystem

- Let's see what a Data-Centric Architecture looks like

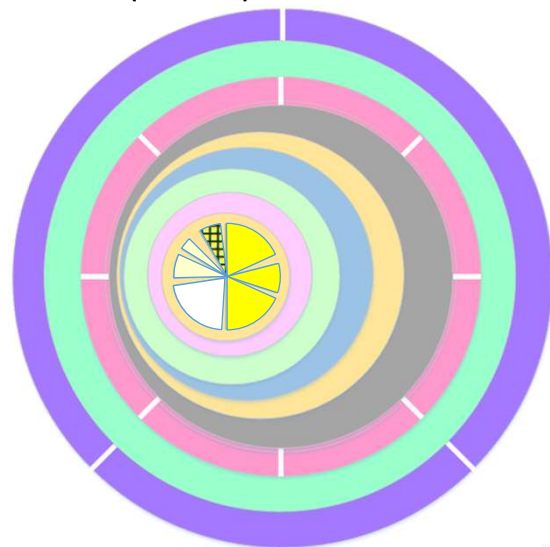
Data-Centric Architecture

21

21

The Data-Centric Architecture (DCA)

- This is an abstract rendering of the layers of the DCA
- Rendered as concentric circles, rather than the usual stacked representation
- We will examine each layer, working our way from the center outward.



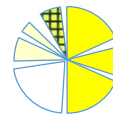
Data-Centric Architecture

22

22

Layer 1 – Data

- The center-most layer represents the data itself
- Wedges symbolize multiple data repositories
 - Not all repos need be natively semantic
- Different colors indicate different sources
- Here, "data" also includes metadata



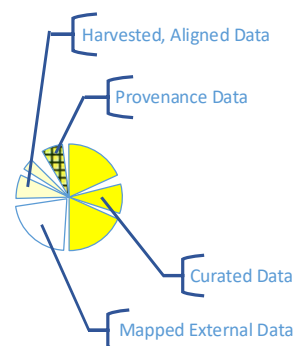
Data-Centric Architecture

23

23

Layer 1 – Data

- Curated data are those data sets the contents of which are under your control
 - Generated by your systems
 - Owned by your organization
- Harvested data are those data sets the contents of which are not under your control, but brought into your repos
 - Purchased or subscription data
- External data do not reside in your repositories, but are accessed remotely
- Provenance metadata about the data sets also sits in your repository



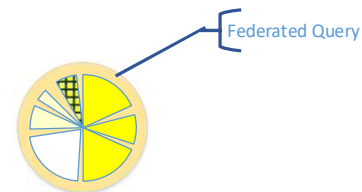
Data-Centric Architecture

24

24

Layer 2 – Federated Queries

- The Federated Query layer represents how all the data are available to query
 - Subject to access controls, of course
- A single query may pull from multiple repositories



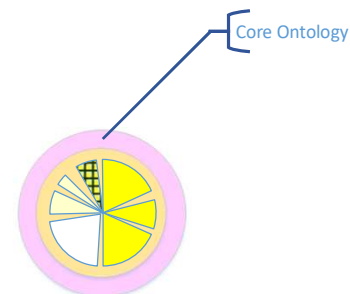
Data-Centric Architecture

25

25

Layer 3 – The Core Ontology

- The Core Ontology layer is where the *meaning* of (most of) the data are described in OWL code
- The core ontology defines the most widely used/shared data in your enterprise
 - Typically ~400 Classes & Properties
 - Typically >100 Categories
- It should be imported into or referenced by other ontologies



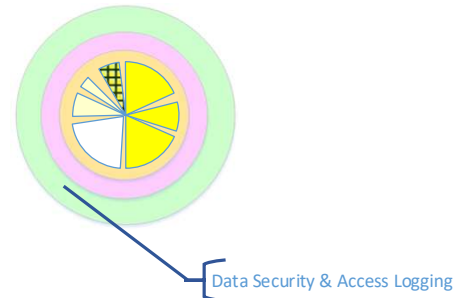
Data-Centric Architecture

26

26

Layer 4 – Data Security

- This layer defines how the Data Layer content is to be protected
- All authorization should be expressed in rules that map to concepts in the core ontology
 - Role-Based Access
 - Repository-Based Access
- All access to the curated data (at a minimum) must go through the security layer
- This layer also defines rules for logging data access information



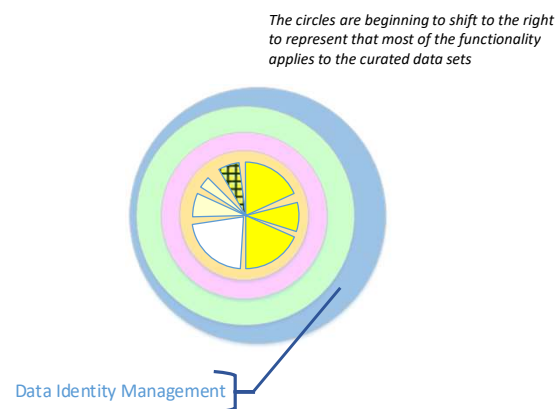
Data-Centric Architecture

27

27

Layer 5 – Data Identity Management

- The Data Identity Management layer contains rules by which we can conclude whether or not a triple's Subject or Object is already known (e.g., it already has an IRI)
- If no known IRI exists, this layer handles minting a new one



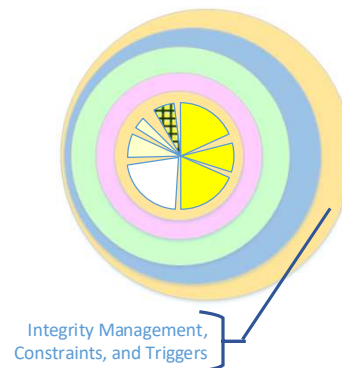
Data-Centric Architecture

28

28

Layer 6 – Integrity, Constraints, Triggers

- This layer begins to introduce more complex behaviors
 - Rules to ensure schema consistency in the datastores
 - Constraints to ensure that only certain values are allowed in certain triples, or to ensure that critical triples are produced
 - Patterns to ensure consistent IRI minting
- These should be defined using the SHACL language



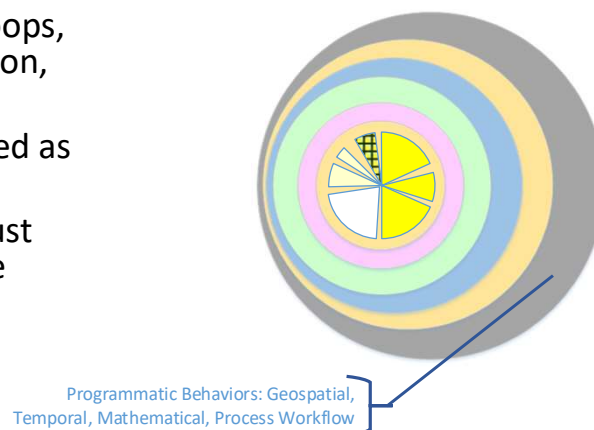
Data-Centric Architecture

29

29

Layer 7 – Programmatic Behaviors

- This layer contains true programmatic behaviors: loops, conditions, data manipulation, calculations, etc.
- Ideally, these are constructed as microservices
- All the code in this layer must conform to and support the layers inside it



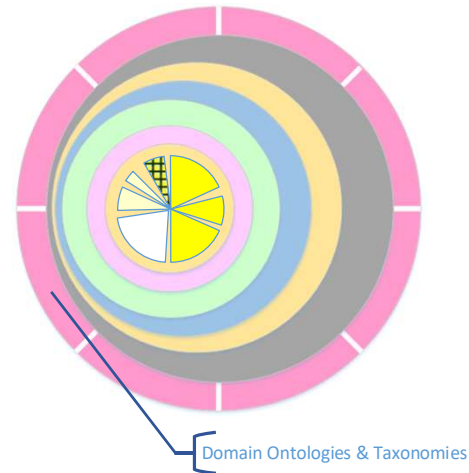
Data-Centric Architecture

30

30

Layer 8 – Domain Ontologies & Taxonomies

- Semantics does not demand a one-model-for-everything approach
- Different domains within the enterprise can have models suited solely to their concerns
 - Sharing classes & properties with other domains, as appropriate
- As a best practice, domain ontologies should import the Core Ontology



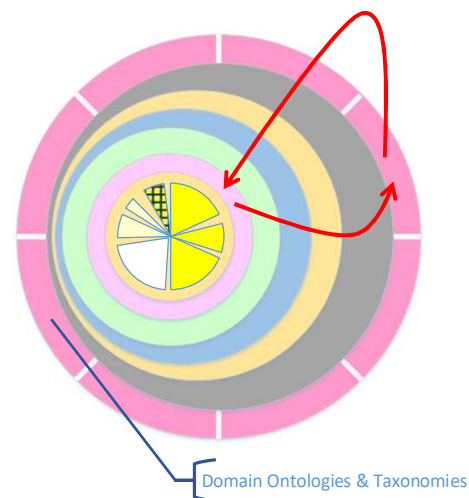
Data-Centric Architecture

31

31

Layer 8 – Domain Ontologies & Taxonomies

- Domain ontologies leverage the core ontology and its supporting layers
- A feedback loop exists to accommodate the needs of domain ontologies

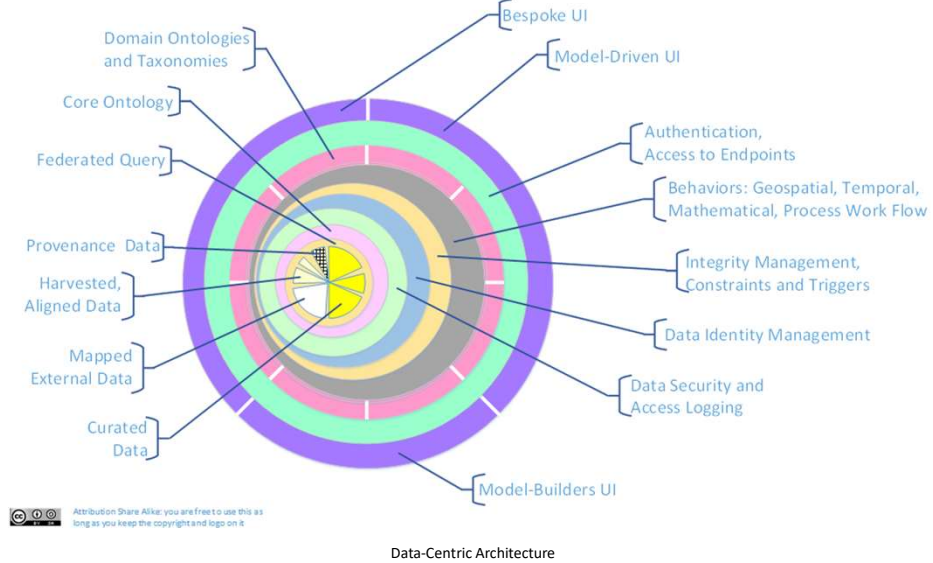


Data-Centric Architecture

32

32

The Data-Centric Architecture



35

35